

VISTA TECNICA

PPAM Scheduler

Guia tecnica ordenada para entender arquitectura, operacion, riesgos y puntos de entrada sin asumir contexto oculto.

Tipo: Aplicacion full-stack

Estado: Borrador basado en evidencia

Archivos analizados: 163

Generado: 4/6/2026, 2:30:19 p.m.

[Abrir vista general](#)

[Ver contexto JSON](#)

Borrador generado a partir de evidencia del repositorio. Completa los campos narrativos antes de compartirlo fuera del equipo.

1. Explicacion tecnica progresiva

El repo combina UI, handlers y logica de negocio en una sola base. ``app/`` contiene superficies web, ``app/api`` expone endpoints protegidos por rol, ``services/`` concentra reglas de dominio y Prisma persiste el estado operativo en PostgreSQL.

Admin y volunteer entran por areas protegidas distintas. La UI envia formularios a route handlers que validan con Zod, revisan sesion/rol y delegan en servicios como ``assignment.service.ts``, donde se recalculan estados, se sincronizan respuestas y se registran actividades o notificaciones.

El entorno local visible usa Next.js en ``localhost:3000``, PostgreSQL por Docker Compose en ``localhost:5432`` y SMTP opcional via Nodemailer. No se observan aun pipelines CI/CD ni manifiestos de despliegue productivo.

La forma correcta de leer este sistema es verlo como monolito web de Next.js: rutas y pantallas en ``app/``, backend HTTP en ``app/api``, reglas del dominio en

`services/` y persistencia en Prisma. El archivo mas importante para comportamiento real es `services/assignment.service.ts`.

Localmente el flujo visible es: levantar PostgreSQL con `npm run db:start`, sincronizar schema con `db:push`, cargar demo users con `db:seed` y arrancar la app con `npm run dev`. El script de DB ya es idempotente cuando el contenedor `ppam-scheduler-postgres` existe de una sesion anterior.

Stack detectado

RUNTIME

Node.js

FRAMEWORKS

Next.js, React

UI

Componentized UI, HTML templates, Lucide Icons, Radix UI, Tailwind CSS

TOOLING

ESLint, Markdown, PostCSS, Prettier, TypeScript, YAML

PRUEBAS

Vitest

DATA

Prisma, Prisma Client

INFRA

Estructura del proyecto

Ruta	Uso aparente	Fuente
app	App Router con login, reset de password y workspaces protegidos para admin y volunteer.	app
app/api	Route handlers para asignaciones, availability, open-slots, puntos, auth, schedule y dashboards.	app/api
components	Componentes de UI y dominio como assignment cards, planner semanal, tablas, modales y formularios.	components
services	Capa principal de negocio para asignaciones, disponibilidad, voluntarios, settings y notificaciones.	services
lib	Auth, constantes, utilidades, validaciones y acceso base a Prisma.	lib
prisma	Schema de dominio, seed y configuracion del cliente de base de datos.	prisma
tests	Pruebas automatizadas, hoy enfocadas en el motor de estados de asignacion.	tests
scripts	Scripts de soporte para levantar, detener y diagnosticar la base local.	scripts

Entrypoints

app/layout.tsx

APP/LAYOUT.TSX

Layout raiz y shell global de la aplicacion.

app/page.tsx

APP/PAGE.TSX

Resuelve el punto de entrada segun sesion y rol del usuario.

app/api/auth/[...nextauth]/route.ts

APP/API/AUTH/[...NEXTAUTH]/ROUTE.TS

Bootstrap de autenticacion por credenciales y sesion.

app/(protected)/admin/schedule/page.tsx

APP/(PROTECTED)/ADMIN/SCHEDULE/PAGE.TSX

Surface principal para preparar semanas, navegar el planner y abrir la creacion de asignaciones.

app/api/assignments/route.ts

APP/API/ASSIGNMENTS/ROUTE.TS

Endpoint central para listar y crear asignaciones desde el workspace admin.

Comandos

Script	Command	Proposito	Fuente
dev	next dev	Inicia el entorno local de desarrollo.	package.json
build	next build	Genera el artefacto de build.	package.json
start	next start	Inicia la aplicacion o servicio.	package.json
lint	next lint	Aplica validaciones de	package.json

Script	Command	Proposito	Fuente
		lint.	
typecheck	tsc --noEmit	Verifica tipos.	package.json
test	vitest run	Ejecuta la suite de pruebas.	package.json
test:watch	vitest	Script observado en el manifest.	package.json
db:start	bash scripts/db-start.sh	Inicia la aplicacion o servicio.	package.json
db:stop	bash scripts/db-stop.sh	Script observado en el manifest.	package.json
db:status	bash scripts/db-status.sh	Script observado en el manifest.	package.json
db:prepare	npm run db:start && npm run db:push && npm run db:seed	Script observado en el manifest.	package.json
db:generate	prisma generate	Script observado en el manifest.	package.json
db:push	prisma db push	Script observado en el manifest.	package.json
db:migrate	prisma migrate dev	Script observado en el manifest.	package.json
db:seed	tsx prisma/seed.ts	Script observado en el manifest.	package.json
postinstall	prisma generate	Script observado en el manifest.	package.json

6. Configuracion, testing y delivery

Entorno

.env, .env.example, .env.local

CI

No se detecto CI visible en el repositorio.

Deploy

No se detectaron artefactos de despliegue visibles.

PRUEBAS

Vitest

UBICACIONES

tests

CANTIDAD VISIBLE

1

Hoy la cobertura automatizada visible es baja y se concentra en `determineAssignmentStatus`. El repo necesita seguir apoyandose en smoke tests manuales de admin y volunteer para cambios funcionales grandes.

DEPENDENCIAS CLAVE

@hookform/resolvers

dependencies ^3.9.0

package.json

@next-auth/prisma-adapter

dependencies ^1.0.7

package.json

@prisma/client

dependencies ^5.22.0

package.json

@radix-ui/react-avatar

dependencies ^1.1.1

package.json

@radix-ui/react-checkbox

dependencies ^1.1.2

package.json

@radix-ui/react-dialog

dependencies ^1.1.4

package.json

@radix-ui/react-dropdown-menu

dependencies ^2.1.4

package.json

@radix-ui/react-label

dependencies ^2.1.1

package.json

@radix-ui/react-scroll-area

dependencies ^1.2.2

package.json

@radix-ui/react-select

dependencies ^2.1.4

package.json

@radix-ui/react-separator

dependencies ^1.1.1

package.json

@radix-ui/react-slot

dependencies ^1.1.1

package.json

@radix-ui/react-switch

dependencies ^1.1.2

package.json

@tanstack/react-query

dependencies ^5.59.20

package.json

@tanstack/react-query-devtools

dependencies ^5.59.20

package.json

@types/bcryptjs

devDependencies ^2.4.6

package.json

@types/node

devDependencies ^22.9.0

package.json

@types/nodemailer

devDependencies ^6.4.17

package.json

@types/react

devDependencies ^19.0.2

package.json

@types/react-dom

devDependencies ^19.0.2

package.json

INTEGRACIONES

PostgreSQL local

La persistencia local usa PostgreSQL por Docker Compose y Prisma.

compose.yaml

NextAuth credentials

La autenticacion se resuelve dentro del repo con NextAuth y adaptador Prisma.

`app/api/auth/[...nextauth]/route.ts`

SMTP / Nodemailer

Las solicitudes y recordatorios de confirmacion se envian por correo y quedan registradas en NotificationLog.

`services/notification.service.ts`

7. Conceptos relacionados y uso especifico

El uso especifico visible es coordinacion semanal de predicacion publica con dos superficies de trabajo: admins que operan la semana y volunteers que responden asignaciones. Eso esta mucho mas claro en las rutas protegidas que en el README corto.

CONCEPTOS TECNICOS RELACIONADOS

- Next.js App Router y route handlers en el mismo repo.
- NextAuth con guards por rol y sesiones enriquecidas.
- Prisma como modelo de dominio y repositorio efectivo.
- State transitions de asignacion segun ventana de confirmacion y respuestas.
- Sincronizacion de notificaciones, responses y assignment activities.

8. Orden de lectura y fuentes clave

LECTURA SUGERIDA

Paso	Archivo	Por que leerlo
1	README.md	Da el mejor contexto funcional disponible para una lectura inicial.

Paso	Archivo	Por que leerlo
2	<code>prisma/schema.prisma</code>	Define el vocabulario del dominio y muestra que el repo no es solo frontend.
3	<code>services/assignment.service.ts</code>	Resume casi todo el comportamiento operativo importante.
4	<code>app/(protected)/admin/schedule/page.tsx</code>	Aterriza el planner y la preparacion de semanas desde la UI.
5	<code>app/(protected)/admin/assignments/page.tsx</code>	Muestra la superficie de seguimiento detallado y acciones de notificacion.
6	<code>components/availability/availability-selector.tsx</code>	Aclara como disponibilidad recurrente y excepciones temporales llegan al flujo real.
7	<code>services/notification.service.ts</code>	Explica el envio real de correos y su trazabilidad en NotificationLog.

FUENTES CLAVE CONSULTADAS

README.md

Fuente primaria para descripcion, objetivo y posicionamiento del proyecto.

`prisma/schema.prisma`

De aqui salen los conceptos mas fuertes del negocio: `scheduleWeek`, `assignment`, `response`, `activeSlot` y `notificationLog`.

services/assignment.service.ts

Es la columna vertebral del dominio: duplicacion de semanas, confirmaciones, vacantes, reemplazos y dashboard.

app/(protected)/admin/schedule/page.tsx

Revela que preparar la semana ya es un flujo visible de UI y no solo capacidad de backend.

app/(protected)/volunteer/confirm/[responseld]/page.tsx

Muestra el acceso directo desde correo para confirmar o rechazar sin navegar por todo el portal.

scripts/db-start.sh

Documenta una decision operativa clave: reutilizar el contenedor existente para que el setup local sea recuperable.

9. Checklist junior y errores comunes

CHECKLIST INICIAL

- Antes de tocar UI, reproduce un flujo real con datos seed: crear semana, crear asignacion, confirmar y cubrir vacante.
- Lee `prisma/schema.prisma` antes de cambiar servicios o validaciones; ahí vive el contrato de dominio.
- Busca reglas en `services/` antes de asumir que una pantalla controla el comportamiento.
- Verifica siempre el rol requerido en `lib/auth/guards.ts` cuando abras o agregues endpoints.

- No des por hecho cobertura automatica amplia; valida typecheck, test y smoke manual.

PITFALLS O CONFUSIONES

- Confundir `app/api` con logica de negocio; los handlers son delgados y el comportamiento importante vive en servicios.
- Asumir que un punto sin active slots esta deshabilitado; en este repo se interpreta como unrestricted temporalmente.
- Olvidar que confirmacion, rechazo y reemplazo deben mantener consistencia entre estado, responses, actividades y NotificationLog.

Observado vs inferido

OBSERVADO

Manifest

package.json

package.json

Descripcion

Production-ready MVP for managing weekly public preaching assignments with role-based admin and volunteer experiences.

README.md

Frameworks

Next.js, React

package.json

UI

Componentized UI, HTML templates, Lucide Icons, Radix UI, Tailwind CSS

package.json

Entrypoints

app/layout.tsx, app/page.tsx

app/layout.tsx

Superficies visibles

(auth) y (protected)

app/(auth)/forgot-password/page.tsx

Archivos de entorno

.env, .env.example, .env.local

.env

Pruebas

1 archivos de prueba detectados

tests

INFERIDO

Tipo de repositorio

medium

Aplicacion frontend

Inferido por frameworks, manifiestos y estructura del arbol.

package.json

Resumen de arquitectura

medium

El repositorio apunta a una aplicacion frontend centrada en React, Next.js, TypeScript, Tailwind CSS.

Inferido a partir del stack detectado.

Riesgos y vacios

- La cobertura automatizada visible sigue siendo baja para un flujo tan operativo; hoy solo hay un test unitario en el motor de estados.
- No se observan CI, despliegue, monitoreo ni politicas de rollback; el camino a produccion no queda demostrado por el repo.
- El acceso directo de confirmacion via `responseld` cumple el MVP, pero conviene revisar endurecimiento antes de un entorno mas expuesto.
- La configuracion SMTP y secretos de sesion pueden romper el piloto silenciosamente si no se validan al levantar el entorno.
- Existe algun pipeline externo, checklist de release o despliegue manual que no este versionado aqui?
- Se espera hardening adicional para los links directos de confirmacion antes de salir de piloto interno?
- Los reportes visibles en sidebar son parte del alcance estable o siguen siendo una capacidad secundaria no critica?